

HR (Heart Rate) Signal

VRET pipeline • Writer: Shayan Itami

1. What this signal is, and why we use it

Heart rate (HR) is the number of heartbeats per minute, derived from the time spacing between successive R-peaks of the ECG (the electrocardiogram). It is the most widely recognised physiological index of autonomic arousal: acute stress, fear, or physical exertion all reliably raise HR through sympathetic acceleration and parasympathetic withdrawal.

In this pipeline, HR is expressed as a **percentage deviation from the subject's own frozen resting baseline**: a positive `delta_hr` means the heart is beating faster than at rest and contributes positively to the stress score. HR carries weight **0.2** in the composite stress score S_t (EDA 0.5, HRV 0.3, HR 0.2) — the smallest weight — because HR is the least specific of the three signals: it rises with physical movement and cognitive load as well as emotional arousal, and its beat-to-beat variability is already better captured by the HRV term.

Compared to HRV and EDA, HR is the **fastest to become available**: it requires only a few heartbeats (~2-3 s at typical rates) and is reported from the very first compute tick onward.

■ **Reference.** *Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996). Heart rate variability: standards of measurement, physiological interpretation, and clinical use. Circulation 93(5):1043-1065 / Eur Heart J 17(3):354-381. — HR as an autonomic index and its relationship to sympathetic activation. Shaffer & Ginsberg (2017). An overview of heart rate variability metrics and norms. Front. Public Health 5:258 — HR norms and arousal context.*

2. The signal path at a glance

Raw ECG (1000 Hz) → **channel-label mapping** (EDA/ECG assignment from LSL stream labels) → **rolling buffer** (deque, capped at 30 s) → **clean** (`nk.ecg_clean`: highpass + 50 Hz powerline removal) → **R-peak detection** (`nk.ecg_peaks` with Kubios correction) → **nk.ecg_rate** (mean over window) → **delta_hr** (% deviation from `avg_hr`) → z-score → S_t .

HR is recomputed every `HR_COMPUTE_INTERVAL` ticks (~0.5 s) using the most recent 30 s of ECG, and is available from the first compute tick once enough beats have accumulated.

■ **Reference.** *Makowski, D., Pham, T., Lau, Z. J., et al. (2021). NeuroKit2: A Python toolbox for neurophysiological signal processing. Behavior Research Methods 53:1689-1696. — The library providing `ecg_clean`, `ecg_peaks`, and `ecg_rate`.*

3. Configuration constants

The values governing the HR computation, quoted from the source:

```
HR_BUFFER_CAP_S    = 30  # at most 30 s of ECG fed to nk.ecg_rate
HR_COMPUTE_INTERVAL = 25  # recompute HR every N ticks (~0.5 s)
HR_CAP_SAMPLES     = HR_BUFFER_CAP_S * SAMPLING_RATE_HZ
HR_SIGMA_FLOOR_PCT = 2.0  # resting HR spread floor in %
```

HR_BUFFER_CAP_S = 30. HR is computed from up to the most recent 30 seconds of ECG. This is a memory cap, not a smoothing window: `nk.ecg_rate` reports an instantaneous rate for each sample in the cleaned ECG, and the mean is taken across however many samples are available up to this cap. 30 s gives plenty of beats (~35-50 at typical resting rates) for a stable mean while keeping the computation lightweight and the estimate current. Unlike the HRV window, the HR cap does not require a fixed minimum before it starts reporting — as soon as two peaks are detected (~2-3 s) a rate can be produced.

HR_COMPUTE_INTERVAL = 25. HR and RMSSD are recomputed every 25 loop ticks. At `LOOP_RATE_HZ = 50` (one tick every 20 ms), this is once every 0.5 s, or approximately 2 Hz. This balances responsiveness against the cost of running `nk.ecg_peaks` on every tick.

HR_SIGMA_FLOOR_PCT = 2.0. The z-score divides the live HR delta by the baseline HR spread (`hr_sigma`). On an unusually flat baseline the measured spread can be implausibly small — as low as 0.25% in one observed run —

which amplifies a trivial 1% HR wobble into a ~16-sigma event, falsely triggering ultra-stressed. Empirical measurement of true beat-to-beat HR variability on the same recording gave 12.4%, while the smoothed windowed-average (what `hr_sigma` measures) gave 2.7%. A 2% floor is physiologically justified: resting HR does not genuinely vary less than ~2% over a 30 s window. The floor only engages on degenerate baselines and prints a [SIGMA-FLOOR] warning.

■ **Reference.** *Task Force (1996) for typical resting HR ranges. The sigma-floor value of 2% is grounded in measured resting HR variability on validation recordings conducted during this pipeline's development; it is flagged as a tunable constant pending more recordings.*

4. Channel mapping and sanity check

Before any HR computation the pipeline must establish which LSL channel carries ECG and which carries EDA. A mis-assignment (the channels swapped) was an early bug that produced impossible RMSSD values (~1000 ms) and a stress score that divided by near-zero. The fix is label-based mapping:

```
# _read_channel_labels() reads the stream's LSL channel labels.
# Map by label name; fall back to fixed index only if labels are absent.
labels = _read_channel_labels(inlet)
for i, lbl in enumerate(labels):
    if 'ECG' in lbl.upper(): ecg_ch = i
    if 'EDA' in lbl.upper() or 'GSR' in lbl.upper(): eda_ch = i
```

Label-based assignment. The fake stream (`fake_opensignals.py`) reads the sensor names from the recording header JSON and labels the LSL channels accordingly — even when the physical sensor order is reversed (EDA,ECG instead of ECG,EDA). The server then maps by label, so a reversed recording is handled transparently without any code change.

After mapping, a beat-plausibility sanity check verifies the assignment is correct:

```
def _beat_plausibility(buf):
    cleaned = nk.ecg_clean(a, sampling_rate=SAMPLING_RATE_HZ)
    _, _info = nk.ecg_peaks(cleaned, sampling_rate=SAMPLING_RATE_HZ,
                           correct_artifacts=True)
    rri = np.diff(_info["ECG_R_Peaks"]) * 1000.0 / SAMPLING_RATE_HZ
    return float(np.mean((rri >= 300) & (rri <= 1500)))

_ecg_plaus = _beat_plausibility(ecg_buffer)
_eda_plaus = _beat_plausibility(eda_buffer)
# Warns loudly if EDA produces more plausible beats than ECG (swap detected)
```

_beat_plausibility(). Cleans and detects peaks in a buffer, then reports the fraction of RR intervals falling in the 300-1500 ms physiological band (40-200 BPM). A real ECG gives a score near 1.0; EDA (a slow, near-DC signal) gives close to 0.0 because its 'peaks' are not heartbeats. If the EDA channel scores higher than the ECG channel, the mapping is almost certainly inverted and a loud warning is printed. This check runs once at baseline start and its output is always visible in the terminal log (the [SANITY] line).

■ **Reference.** *Task Force (1996) for the 300-1500 ms physiological RR range. Makowski et al. (2021) for `nk.ecg_clean` and `nk.ecg_peaks`.*

5. Baseline computation (the frozen reference)

During the 120 s baseline the pipeline computes `avg_hr` — the subject's resting heart rate — and establishes the per-signal mean and sigma used for z-scoring.

Step A — ECG cleaning and peak detection.

```
ecg_all = nk.ecg_clean(ecg_all_raw, sampling_rate=SAMPLING_RATE_HZ)
peaks_all, info_all = nk.ecg_peaks(ecg_all, sampling_rate=SAMPLING_RATE_HZ,
```

```

        correct_artifacts=True)
ecg_q = float(np.nanmean(nk.ecg_quality(ecg_all,
        rpeaks=info_all["ECG_R_Peaks"],
        sampling_rate=SAMPLING_RATE_HZ)))

```

nk.ecg_clean first. nk.ecg_peaks expects a cleaned signal; raw mains-contaminated ECG (50/60 Hz powerline interference) yields zero or spurious R-peaks. The highpass filter in ecg_clean removes the DC offset and low-frequency baseline wander; the powerline notch removes mains interference. This is the canonical NeuroKit order: ecg_clean → ecg_peaks.

correct_artifacts=True. Runs the Kubios beat-correction method (Lipponen & Tarvainen, 2019) on the detected peaks, adjusting missed, extra, and ectopic beats before the rate is computed.

nk.ecg_quality. Returns a 0-1 signal-quality score for the ECG channel. Printed in the freeze block so a poor-quality baseline (e.g. 0.74) is immediately visible. Low quality implies unreliable R-peaks and therefore unreliable HR and HRV.

Step B — frozen avg_hr.

```

avg_hr = float(np.nanmean(nk.ecg_rate(peaks_all, sampling_rate=SAMPLING_RATE_HZ)))

```

nk.ecg_rate. NeuroKit's official heart-rate function. It converts the R-peak index series into an instantaneous rate signal (BPM) for every sample by interpolating between detected beats. The mean across all baseline samples gives avg_hr — the resting heart rate frozen at baseline end. Every live HR measurement is compared against this single number.

Step C — baseline sweep: collecting hr_mean and hr_sigma.

```

# inside the 60s-step baseline sweep:
hr_start = max(0, end - HR_CAP_SAMPLES)
sig_h, ih = nk.ecg_peaks(ecg_all[hr_start:end],
        sampling_rate=SAMPLING_RATE_HZ, correct_artifacts=True)
hh = float(np.nanmean(nk.ecg_rate(sig_h, sampling_rate=SAMPLING_RATE_HZ)))
_raw_hr_d.append((hh - avg_hr) / avg_hr * 100) # HR % deviation
...
hr_mean = float(_raw_hr_d.mean())
hr_sigma = float(_raw_hr_d.std())
hr_sigma = max(hr_sigma, HR_SIGMA_FLOOR_PCT, SIGMA_FLOOR) # floor 2%

```

The sweep logic. A 60 s sliding window steps across the baseline. At each position HR is recomputed from the most recent 30 s of ECG (the same cap used live), expressed as a percentage deviation from avg_hr, and appended to _raw_hr_d. This collection represents how much HR naturally wanders during rest.

hr_mean and hr_sigma. The mean and standard deviation of HR% deviation across all baseline windows. hr_mean captures any systematic drift (typically near 0%); hr_sigma captures the natural resting spread. Both are frozen and used to z-score every live HR delta. The sigma floor (HR_SIGMA_FLOOR_PCT = 2.0%) prevents a flukey flat baseline from producing a near-zero sigma that would blow up the z-score.

■ **Reference.** Task Force (1996) for resting HR conventions. Lipponen & Tarvainen (2019) for the Kubios correction applied in ecg_peaks. The 2% sigma floor is an engineering guardrail set from empirical validation on this pipeline's recordings.

6. Live computation

During the live phase the ECG buffer grows tick by tick. Every HR_COMPUTE_INTERVAL ticks (~0.5 s) the pipeline re-runs the full clean → detect → rate sequence on the most recent 30 s:

```

ecg_for_hr = ecg_arr[-HR_CAP_SAMPLES:] if len(ecg_arr) > HR_CAP_SAMPLES else ecg_arr
ecg_for_hr = nk.ecg_clean(ecg_for_hr, sampling_rate=SAMPLING_RATE_HZ)
peaks_hr, info_hr = nk.ecg_peaks(ecg_for_hr,
        sampling_rate=SAMPLING_RATE_HZ, correct_artifacts=True)

```

```
rate = nk.ecg_rate(peaks_hr, sampling_rate=SAMPLING_RATE_HZ)
hr_bpm = float(np.nanmean(rate))
if not np.isfinite(hr_bpm):
    hr_bpm = None
```

Cap slice (HR_CAP_SAMPLES). Only the last 30 s of the ECG deque is passed to the peak detector. This keeps both processing time bounded and the HR estimate current: a heart-rate change 40 s ago should not dilute the present reading.

Same clean → peaks → rate sequence as baseline. Using the identical pipeline in both phases ensures the live HR and the frozen avg_hr are produced the same way, so their comparison is meaningful.

Finite check → None. If nk.ecg_rate produces a non-finite value (e.g. no peaks detected on a very noisy segment), hr_bpm is set to None. A None HR is excluded from the score composition that tick — the remaining signals (EDA, HRV) carry on with their weights renormalised so S_t stays on the same scale.

The HR delta and how it flows into the score:

```
delta_hr = (hr_bpm - avg_hr) / avg_hr * 100 if hr_bpm is not None else None
```

delta_hr = (hr_bpm – avg_hr) / avg_hr × 100. Percentage deviation of current HR from the frozen resting baseline. Positive means the heart is beating faster than at rest (arousal). Negative means HR is below baseline (calmer than rest). Using a percentage rather than raw BPM means the scale is comparable across subjects with different resting rates: a 10% rise means the same relative effort whether the resting rate is 60 or 90 BPM.

HR is available from the first compute tick (after ~2-3 s of data). Unlike HRV there is no bootstrap period: as soon as at least two R-peaks have been detected, nk.ecg_rate can produce a value. The [LIVE] log line shows HR=— only for the very first few seconds before the buffer holds enough beats.

■ **Reference.** Task Force (1996) for the HR percentage-deviation convention. Makowski et al. (2021) for nk.ecg_rate. Lipponen & Tarvainen (2019) for Kubios correction.

7. How HR enters the composite score

This section is common to all three signal documents so each stands alone. Each signal is converted to a z-score against its own frozen baseline mean and spread, then weighted:

```
def zscore_s_t(eda_phasic_v, hr_pct, hrv_pct):
    zE = (eda_phasic_v - eda_mean) / eda_sigma
    zH = (hr_pct - hr_mean) / hr_sigma
    zV = (hrv_pct - hrv_mean) / hrv_sigma
    return WEIGHT_EDA*zE + WEIGHT_HRV*zV + WEIGHT_HR*zH # 0.5 / 0.3 / 0.2
```

The HR z-score zH = (delta_hr – hr_mean) / hr_sigma. Expresses the current HR deviation in units of the subject's own resting HR spread. Because all three signals are z-scored before weighting, a 5% HR rise and a 5% HRV suppression can be meaningfully compared and combined — they are both expressed in the same 'standard deviations from this person's baseline' language. Without z-scoring, EDA (in microsiemens) and HR (in percent) would be incommensurable, and the signal with the largest raw numbers would dominate regardless of the 0.5/0.3/0.2 weights.

When HR is None. If hr_bpm could not be computed that tick (no valid peaks), the HR term is excluded from the sum and the remaining weights are renormalised:

```
terms = []
if delta_eda is not None:
    terms.append((WEIGHT_EDA, (delta_eda - eda_mean) / eda_sigma))
if delta_hrv is not None:
    terms.append((WEIGHT_HRV, (delta_hrv - hrv_mean) / hrv_sigma))
if delta_hr is not None:
    terms.append((WEIGHT_HR, (delta_hr - hr_mean) / hr_sigma))
```

```
wsum = sum(w for w, _ in terms)
s_inst = sum(w * z for w, z in terms) / wsum * (WEIGHT_EDA + WEIGHT_HRV + WEIGHT_HR)
```

Dividing by wsum and multiplying by the total weight (1.0) keeps S_t on the same absolute scale whether HR is present or not, so the thresholds remain valid.

The instantaneous score is smoothed over a rolling buffer, then classified into three bands against thresholds derived from the baseline score distribution:

```
if s_t >= thresh_high: state = "ultra-stressed"; unity_cmd = "decrease"
elif s_t >= thresh_mild: state = "stressed"; unity_cmd = "neutral"
else: state = "calm"; unity_cmd = "increase"
```

Below the lower threshold the patient is calm and the VR balloon ascends (exposure advances); between the thresholds it holds; above the upper threshold the balloon descends (exposure backs off). Thresholds sit at 1.28σ (mild, ~90th percentile of baseline) and 2.33σ (high, ~99th percentile) above the baseline score mean.

■ **Reference.** *The weighted-sum fusion and the 0.5/0.3/0.2 weights are a design decision of the lab/pipeline. The renormalisation when a term is missing is an engineering choice to maintain score scale. The 1.28/2.33 sigma thresholds correspond to the 90th and 99th percentiles of a normal distribution (Task Force, 1996; standard statistical convention).*

8. Known limitations and failure modes

- **HR is the least specific arousal marker.** Heart rate rises with movement, cognitive load, and temperature as well as emotional arousal. It carries the lowest weight (0.2) precisely for this reason; the EDA and HRV terms are more specific to the autonomic responses relevant to phobia exposure.
- **Sigma floor and baseline flatness.** The `hr_sigma` is measured from the 30 s windowed-average HR, which is inherently smoother than beat-to-beat HR. On unusually flat baselines the measured sigma can be implausibly small (0.25% was observed), amplifying trivial wobbles into large z-scores. The 2% floor addresses this, but the floor value itself is based on a small number of validation recordings and should be revisited as more data is collected.
- **No artifact correction on the rate itself.** Kubios corrects the peak positions but does not flag or remove the resulting HR estimate if it is implausible (e.g. a brief spike to 200 BPM from one spurious peak). The 30 s window mean dampens single-beat spikes considerably, but does not eliminate them.
- **Band placement is not yet validated.** The pipeline correctly separates aroused from calm recordings, but whether the current thresholds correspond to clinically meaningful arousal levels requires a controlled graded-arousal recording for calibration.
- **Rate mismatch with non-1000 Hz recordings.** The pipeline assumes the ECG is sampled at `SAMPLING_RATE_HZ = 1000` Hz. If a recording at a different rate (e.g. 200 Hz) is used with the fake stream, the time axis is compressed and HR values are compromised. The pipeline currently has no automatic rate detection; all recordings must be 1000 Hz.

9. References

- [1] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology (1996). Heart rate variability: standards of measurement, physiological interpretation, and clinical use. *Circulation* 93(5):1043-1065 / *Eur Heart J* 17(3):354-381. — HR as an autonomic index, resting HR norms, and standard threshold conventions.
- [2] Lipponen, J. A., & Tarvainen, M. P. (2019). A robust algorithm for heart rate variability time series artefact correction using novel beat classification. *Journal of Medical Engineering & Technology* 43(3):173-181. — The Kubios method invoked by `nk.ecg_peaks(correct_artifacts=True)`.
- [3] Makowski, D., Pham, T., Lau, Z. J., et al. (2021). NeuroKit2: A Python toolbox for neurophysiological signal processing. *Behavior Research Methods* 53:1689-1696. — The library providing `ecg_clean`, `ecg_peaks`, `ecg_rate`, and `ecg_quality`.
- [4] Shaffer, F., & Ginsberg, J. P. (2017). An overview of heart rate variability metrics and norms. *Frontiers in Public Health* 5:258. — HR norms and sympathetic arousal context.